
SharpeArena: The Point-in-Time (PIT) RL Environment for Trading Agents

Tiberiu Toca
General Liquidity
tibi.toca@gmail.com

Abstract

SharpeArena is a point-in-time reinforcement-learning environment for trading agents whose interface excludes future-bar access, whose Rust core is pinned byte-for-byte across native, WebAssembly and Python bindings, and whose trajectories are recomputed from recorded decisions. Agents use a governed JSON contract; procedural scenarios occupy disjoint train, gap and held-out seed bands; ranking is delegated to SharpeBench. The experiments calibrate these instruments rather than make market claims. The canonical generator fails its own realism gate on 22 of 24 seeded panels; an opt-in volatility-persistence driver improves clustering, while the finite-panel Fano statistic is now reported only as exploratory because its null is not calibrated. Fixed-spread market making exhibits a U-shaped sampled-grid regret curve against the Avellaneda–Stoikov reference; its minimum is not statistically distinguishable from the reference at 16 episodes, which is not evidence of equivalence. A normalized-flow concave-impact ablation makes the manipulation probe falsifiable. An exploratory 135-cell asymmetric search finds a best no-follower cell of $+21.8 \times 10^{-4}$; its pointwise 95% interval $[17.4, 26.2] \times 10^{-4}$ remains positive after a Bonferroni familywise correction, $[10.0, 33.6] \times 10^{-4}$. All 45 sampled linear-impact cells have negative means, a finite-grid result rather than a universal claim. An honest autoregressive adversary remains near its unconditional baseline, while a public 2^{16} seed band is recovered 16/16 times; an opt-in sealed derivation defeats that specific scanner 0/16 when its salt is withheld, and the companion forward protocol can commit the salt hash before entry. An out-of-band oracle witness, using common random numbers and monotonicity checks, shows that eligibility is attainable on every tier and that the every-seed PSR conjunction is last to open at all ten observed thresholds. Finally, an eight-seed ecology replication overturns its original single-seed headline: shock-driven winner replacement occurs in one seed, not seven.

1 Introduction

An eval is useless without an environment. The companion SharpeBench paper [Toca, 2026] argues that ranking trading agents on raw return over short windows measures luck, and supplies a deterministic scoring kernel that deflates for search breadth [Bailey and de Prado, 2014], gates on per-run reliability [Yao et al., 2024], and zeroes entries that bypass risk controls. But a scorer consumes trajectories, and every one of its guarantees is conditional on the trajectory being honest. Three failures upstream of scoring void any score. An agent that observes one future bar has an edge no deflation can remove; look-ahead bugs are a pervasive and usually invisible failure mode of backtests, alongside the selection bias the deflation literature centers [Bailey and de Prado, 2014]. An

environment that is not bit-reproducible cannot support a leaderboard, because the same submission scores differently on different machines and no one can check anyone else’s number. And a trajectory that cannot be recomputed from the environment that allegedly produced it is a self-reported claim, not evidence.

SharpeArena is the producer built against those three failures, and its design method is uniform: exclude each failure at the environment’s own interface, then add a detection layer behind the structural guarantee for defense in depth. Look-ahead is prevented by the shape of the data layer, which has no API that returns a future bar, and additionally policed by a deny-list guard at the harness boundary; this is an interface guarantee, and Section 7 states what it does not cover, including in-band prediction of the public deterministic generator. Determinism is achieved by restricting the scenario generator’s arithmetic to operations that are exact and identical on every platform, and additionally pinned by committed golden hashes checked in continuous integration on the native and WebAssembly builds. Trajectory honesty is achieved by recording only decisions and recomputing everything else, and additionally exercised by a test that doctors a trajectory and asserts the replay diverges.

The interface follows the pattern that made Gym the substrate of a decade of reinforcement-learning research [Brockman et al., 2016, Towers et al., 2024]: a small, stable boundary that outlives any single environment behind it. An agent is a program in any language that reads a JSON `Observation` and writes a JSON `Decision`. The contract is versioned, additive-only, schema-published and governed by a written deprecation policy, so a vendor can build against it once. Conform to the contract and you are scorable everywhere the SharpeBench kernel runs. Whether an ecosystem forms around this contract is an adoption question the paper does not answer; Section 7 records that no third party has yet submitted an agent.

Scope and contribution boundary. This paper claims the environment and its protocol: the interface-level leak-freedom, the cross-runtime determinism, the replay verifiability, the wire contract, the seed-band generalization protocol, and the probe layer, together with the calibration experiments that exercise them. The scoring semantics, the deflation prior and the unit bug in the pre-0.5.0 kernel are the companion SharpeBench paper’s contributions [Toca, 2026]; F1 appears here only as validation that the producer-scorer separation catches scorer bugs. No agent is trained in this paper: every experiment uses fixed reference policies, so the results calibrate the instruments rather than demonstrate learned performance, and a learning baseline through the shipped Gymnasium adapter is named future work in Section 7.

Contributions. (1) A PIT RL environment for trading agents that is leak-free at its interface boundary, cross-runtime byte-deterministic in its Rust core, and replay-verifiable, with the enforcing code identified for each property. (2) A governed agent wire contract at version 1.0 with JSON Schemas, conformance fixtures and an additive-only evolution policy. (3) A procedural scenario generator with disjoint train, gap and held-out seed bands in the Procggen tradition [Cobbe et al., 2020], making generalization a measured quantity. (4) A task suite that includes a market-making environment shipping the analytical Avellaneda–Stoikov quoting policy [Avellaneda and Stoikov, 2008, Guéant et al., 2013] as a reference, so agents are scored on regret against a fixed analytical baseline rather than against each other. (5) A diagnostic layer that probes realism, manipulation payoff boundaries, adverse-selection markouts, population ecology and generator predictability, including an opt-in concave-impact path that leaves the canonical linear golden path byte-identical, and an opt-in sealed-seed derivation that hides the held-out seeds behind a commit-reveal salt while keeping the disjointness proof public. (6) Ten executed experiments and three follow-ups (Section 5) in which every number is produced by a committed script listed in Section A.

What the evidence shows. The findings are instrument calibrations on canonical tape that fails the calibrated realism conjunction in 22 of 24 seeded panels (Section 5.5). Fixed-spread market making has a U-shaped sampled-grid regret curve whose minimum is unresolved from the analytical reference at 16 episodes (Section 5.3). Cross-regime transfer costs the Calm-selected reference

nearly its entire DSR (Section 5.4). The symmetric manipulation schedule remains negative under normalized-flow concavity; an exploratory 135-cell asymmetric search finds two selected $\kappa = 0.5$ cells whose Bonferroni familywise intervals remain positive, while all 45 sampled linear means are negative (Sections 5.7 and 5.8). A causal AR adversary remains near its unconditional baseline, whereas a public 2^{16} seed band is recovered 16/16 times (Section 5.12); withholding a high-entropy salt drops that specific scan to 0/16 in a simulated commit–reveal workflow (Section 5.13). The adverse-selection, failure-taxonomy and replicated ecology probes expose their own limitations rather than hiding them (Sections 5.9 to 5.11). No reference policy is rank-eligible, yet a common-random-number oracle witness attains eligibility on every tier and finds the every-seed PSR conjunction last to open at all ten monotone thresholds (Section 5.2).

2 Design principles

Every decision in the environment traces to one of seven principles, stated here so the rest of the paper reads as their consequences.

Leak-free at the interface. The environment owns the time cursor, and the data layer exposes no operation that returns a bar after it. An agent cannot peek through the interface because there is nothing to call, not because a rule forbids it. Detection layers exist, but they sit behind the structural guarantee, never in place of it. This is a guarantee about the interface’s shape, not about information: what an agent might infer in-band from the observations the interface does return, given a public deterministic generator, is a separate question, treated in Section 7.

Deterministic across runtimes. A published number must be byte-identical on any platform and any surface. The determinism-critical core is one Rust engine; its scenario transform uses only multiply, add, divide and max, because $\ln$ and $\exp$ differ across libm implementations. Golden hashes committed in the source pin the outputs, and the WebAssembly build asserts equality with the native engine rather than assuming it.

Recompute, do not trust. A trajectory is evidence only if a third party can recompute it. Runs record the agent’s decisions and nothing derived; replay against the frozen scenario reproduces returns byte-for-byte, and a tampered trajectory reproduces something else.

The contract is the product. The agent interface is a tiny JSON surface that evolves additively under a written governance policy, with schemas and conformance fixtures as the authoritative artifacts. An interface that moves under its implementers cannot be built against; stability under governance is what makes the contract worth conforming to.

Generalization is measured, not presumed. Scenarios are procedural functions of an integer seed, in the Progen model [Cobbe et al., 2020], and the protocol fixes disjoint train, gap and held-out seed bands. Overfitting becomes one number, the train-minus-test deflated Sharpe, instead of a suspicion.

The environment produces; the kernel judges. SharpeArena computes no rank. Scoring is delegated to the SharpeBench kernel [Toca, 2026], so the deflation, reliability and process gates are specified once, in one place, and the environment cannot quietly disagree with the scorer about what a good run is.

Distrust the simulator too. A synthetic market that pays manipulation without bound, or that lets makers profit from informed flow, or that emits Gaussian tape, is an answer key for the wrong exam. The probe layer treats the simulator as a subject: realism certification against the stylized facts [Cont, 2001], payoff-boundary sweeps for manipulation, markout decomposition for adverse selection, and ecology runs that ask which strategies survive contact with each other. At this revision the probes report findings against the environment as often as for it: the realism gate fails the canonical tape (Section 5.5) and the adverse-selection levels sit on the wrong side of this principle’s own standard (Section 5.9).

One consequence is worth stating before the experiments. The first three principles are properties of committed code, checkable by the listed tests. The empirical claims are properties of committed

evidence artifacts and their producer scripts in the same Git revision. F1 serializes package version 0.9.0 and the legacy files record SharpeBench 0.5.0 where stated; several other legacy JSON files did not serialize a runtime version, so the paper does not infer one. The current F4, F5 and witness artifacts were regenerated after their estimator corrections. All findings calibrate instruments on this generator rather than certify a market property.

3 The environment

SharpeArena is a Rust workspace of three crates. The crate `sharpearena` holds everything determinism-critical: the stepping engine, the wire contract, the scenario generator, the limit-order-book matching engine, mandates and the execution-noise model, built on the published SharpeBench engine crates rather than a vendored copy. The crate `sharpearena-wasm` compiles the same kernel to WebAssembly and tests it against the native engine. The crate `sharpearena-py` binds the engine into a Python package that adds the ecosystem adapters: Gymnasium [Towers et al., 2024], PettingZoo [Terry et al., 2021], Minari [Farama Foundation, 2024], a `verifiers` training environment, and an MCP server. The adapters are thin by policy; anything that must be byte-identical lives in Rust.

3.1 Point-in-time observation, and the guard behind it

The engine owns the time cursor. On each step it emits a `MarketObservation` whose per-symbol snapshot carries a trailing window of closes up to the decision date, with optional fundamentals and news channels derived from that same trailing window. There is no API on the data layer that returns a bar after the cursor, so a policy cannot observe a future bar: the property is the shape of the interface, not a rule. It is also only a property of the interface: it bounds what can be requested, not what can be inferred from the trailing window a request returns (see Section 7). A property test iterates every scenario tier and asserts that no surfaced window extends past the cleared bar and that each window is a suffix of the true history.

One layer out, at the agent-harness boundary, sits `LookaheadGuard`, a refusal layer for tools and policies that try to read what the environment will not give. It carries a deny-list of named operations, each mapped to the reason it is refused: reading the raw dataset (the answer key), slicing past the current index, peeking the next bar, or feeding the scenario seed to the policy as a feature, which would identify the episode and trivialize generalization. Substring tells catch novel operation names, and an index check bounds any point-in-time slice: reading an index strictly greater than the current bar raises. A research escape hatch exists as an explicit environment variable, so relaxing the guard is a recorded decision rather than an accident. The guard is defense in depth behind the structural guarantee: a harness that bypasses it still cannot make the environment leak.

3.2 Determinism across runtimes

A scenario is a pure function of one 64-bit seed. The generator’s price transform uses only multiply, add, divide and max; it never calls `ln` or `exp`, which differ across libm implementations, so a generated panel is byte-identical on any platform and any runtime. The commitment is pinned, not assumed: the source commits FNV-1a fingerprints of canonical generated scenarios and of a scripted order sequence’s fill tape. Tests recompute the hashes on every commit, and WebAssembly parity tests require native-identical returns, traces and scenarios. The generator is deliberately public and deterministic; that makes the hashes checkable and future bars computable from a recovered seed. Section 5.12 measures the boundary: a causal AR adversary finds no undisclosed stressed-tier edge, but a public bounded band of 2^{16} seeds is exhaustively inverted from one observed bar in about one second. Held-out evaluation therefore requires high-entropy unpublished seeds, not merely disjoint public bands.

3.3 Recompute-from-decisions tamper evidence

A rollout trace is a versioned JSONL artifact that records the agent’s decisions and rejects anything leaky: the writer refuses values that would embed future data in the record. Everything else, returns included, is recomputed at verification time by replaying the decisions against the frozen scenario, and because the engine is deterministic the recompute is byte-identical. The npm package’s test suite exercises the adversarial case directly: it captures a trajectory, doctors every order’s target weight, replays both, and asserts the tampered artifact cannot reproduce the honest returns. An agent cannot lie about its performance to anyone willing to run the replay.

3.4 Procedural scenarios and disjoint seed bands

Scenario families follow Procgen’s integer-seed-interval model [Cobbe et al., 2020]: Calm, Hard and Extreme volatility-and-jump tiers, a cointegrated-pairs family with a genuinely mean-reverting spread, and a regime-shift family that hands a trend regime over to a whipsaw. The Rust core exposes `train_test_split`, which carves a disjoint test family (two separated integer intervals, with the disjointness asserted in tests) from a bounded train interval and refuses an unbounded one, and `cross_regime_split`, which holds the seed band fixed while swapping the regime, a strictly stronger robustness probe than a within-tier split. The evaluation protocol fixes the canonical bands: train seeds $[0, 256)$, a gap of ten thousand seeds that is never sampled, and a held-out test band $[10256, 10512)$. The Python layer additionally reserves an absolute evaluation namespace starting at seed 10^6 and commits a named held-out regression set inside it, with a disjointness guard asserting the set never touches the train band. Overfitting is then a measurement: `generalization_gap` scores the same policy on train and held-out bands and reports the differenced deflated Sharpe, and `cross_regime_transfer` reports the in-distribution minus zero-shot out-of-distribution gap over identical seeds, which is zero by construction when the regimes coincide.

3.5 The task suite and the closed-form reference policy

Beyond the canonical position-trading environment, the suite ships a simplex-allocation portfolio task, a VWAP/TWAP implementation-shortfall execution task, and two market models with genuine microstructure: an endogenous shared-book market where aggregate agent flow moves the cleared price through a Kyle permanent-impact term [Kyle, 1985] and an Almgren-Chriss temporary-impact term [Almgren and Chriss, 2001], and a deterministic limit-order-book market with integer ticks, price-time priority, a single-price call-auction uncross and a read-only walk-the-book sweep-cost query.

Both market models clear once per step at a single price, so the trading mechanism is closer to a frequent batch auction [Budish et al., 2015] than to a continuous-time limit order book. The competition margins that continuous markets create, latency races, queue-position value, sniping of stale quotes, do not exist in the environment, and no maker-taker fees or rebates are modeled: every Sharpe, regret and markout in Section 5 is gross of fees. The restriction is deliberate, it keeps clearing deterministic and byte-reproducible, and it makes the environment a batch-auction laboratory rather than a latency one, but it is a scope restriction and results should be read inside it.

The market-making task is the suite’s calibration instrument. It implements the Avellaneda-Stoikov model [Avellaneda and Stoikov, 2008] and ships the model’s closed-form quoting policy beside the environment: reservation price $r = s - q\gamma\sigma^2\tau$ and half-spread $\delta^* = \gamma\sigma^2\tau/2 + \gamma^{-1} \ln(1 + \gamma/\kappa)$, skewed by inventory. This closed form is the source model’s asymptotic approximation, not an exact optimum: the exact treatment of the same problem is given by Guéant et al. [2013], and no proof is offered that the policy is optimal for the implemented reward, which adds an inventory cap, a running inventory penalty and a terminal liquidation charge the source model lacks. We therefore call it the closed-form reference policy. Both the environment and the policy are pure functions of the same frozen parameter set, so the regret metric `mm_regret` runs the candidate and the reference on identical seeds and reports the mean reward gap; the reference scores zero against itself by construction, which fixes the metric’s zero point without claiming the zero point is unbeatable. The model also assumes

fills that do not select against the maker, which sits in deliberate tension with the adverse-selection probe of Section 5.9. Gym-style market-making environments built on the same model family exist [Jerome et al., 2023]; what this task adds is the shipped reference policy and the regret-not-PnL scoring convention inside the verifiable-producer stack.

3.6 Mandates and deferred claims

Each scenario can sample a trading mandate, long-only, market-neutral, drawdown-capped or beat-a-benchmark, and the objective is conditioned on it: `mandate_breach` detects structural violations from the realized weights and returns, and the training rubric penalizes wrong-objective behavior rather than merely not rewarding it. A deterministic failure taxonomy classifies every episode into a single worst-case-first disposition, cascade-wiped before bankrupt before stopped-out before mandate breach before clean, and a suite-level rollup tallies the distribution with a schema that always carries every mode.

Questions that resolve after the episode get the same structural treatment as look-ahead. A `DeferredDesk` is the only object the agent touches: its entire state is a list of claims and one integer clock advanced by the harness, it holds no dataset and no environment reference, and it computes no score, so there is no code path from a claim to its resolving datum. Resolution is a free function over claims and outcomes that refuses any outcome available at or before the claim’s commit bar and any outcome predating the stated horizon. The agent cannot see the answer at commit time because the object it holds cannot produce one.

3.7 The probe layer: distrusting the simulator

A synthetic market can flatter agents in ways real markets do not, so the suite probes itself.

Realism. A stylized-facts battery computes excess kurtosis, absolute-return autocorrelation, gain/loss skew and aggregational Gaussianity in the sense of the classical catalogue [Cont, 2001], plus a timescale-asymmetry statistic in the sense of Zumbach [2009] and an exploratory Fano exceedance-count ratio. `certify_realism` gates only three calibrated directional facts: kurtosis, absolute-return autocorrelation and aggregational Gaussianity. Fano is reported but ungated because its threshold is estimated from the same short panel and its finite-sample null is not one. Certification is reported beside the results rather than required for the leaderboard, and the canonical configuration still fails it on 22 of 24 seeded panels (Section 5.5). The intended endgame is a generator revision that passes on disjoint calibration seeds or leaderboard claims conditioned on certification; until then the gate keeps the failure visible.

Manipulation. A crude push-and-unwind schedule, the classic trade-based manipulation of Allen and Gale [1992], is scored against its own zero-impact reference: the live and reference runs share the seed and follower population, and differ only in impact. A boundary sweep varies one parameter and a size-response sweep asks whether payoff is bounded. Linear permanent impact makes the baseline a model-consistency check under the assumptions of Huberman and Stanzl [2004], Gatheral [2010]. On separate entry points, nonlinear permanent impact is $\lambda \text{sign}(Q/V)|Q/V|^\kappa$, with dimensionless flow Q/V ; $\kappa = 1$ uses the exact original linear arithmetic. Section 5.7 evaluates the symmetric schedule, and Section 5.8 reports a 135-cell exploratory asymmetric search with selection-adjusted inference.

Adverse selection. Informed meta-order flow is routed against a maker roster, with fills struck at the pre-move mid so the maker’s markout carries the informed trader’s edge. The mechanism the probe operationalizes is the classical one [Glosten and Milgrom, 1985], with one deliberate simplification: the price path is exogenous and quotes do not widen in equilibrium response to toxicity, so the probe measures markout accounting against drift, not equilibrium adverse selection. The design is a paired control: the informed leg sides child orders on the alpha, the uninformed leg sides the identical flow on a coin while holding the price path fixed, and the comparison reports mean markout per filled unit at each horizon. If the informed leg does not mark out worse, the module’s own numbers are declared noise.

Ecology. A replicator dynamic in the market-ecology tradition [Farmer, 2002, Lux and Marchesi, 1999] runs a population of strategies over generations: fitness earned in a shared-book market where a species that gains share becomes a larger fraction of the flow everyone else fills against, seat allocation from population shares, extinction below a threshold, an innovation operator that breeds parameter-perturbed variants, and shock schedules that rotate regimes or scale impact to simulate liquidity droughts. The control schedule holds the environment steady, so any claim about a shock has a baseline to beat.

4 The agent contract and its governance

An agent is an external program in any language that reads a `MarketObservation` as JSON and replies with a `Decision` as JSON. The harness sends an observation, the agent returns a decision, repeat. Everything else in this paper exists to make that loop trustworthy; this section is the loop's promise of stability.

4.1 Authoritative artifacts

The contract has four authoritative artifacts, all committed. `CONTRACT_VERSION`, currently 1.0, is the single source of truth for the wire version and lives in the contract module of the core crate. Two JSON Schemas, draft 2020-12, define `MarketObservation` and `Decision` for non-Rust implementers. A conformance kit of fixture documents, covering the normal multi-symbol case, a single symbol, an empty-portfolio start, signed-weight shorting and a legacy decision shape, is exercised by a committed conformance test. And a governance document states the evolution rules in writing.

4.2 Additive-only evolution

The contract evolves additively. The only backwards-compatible change is a new field that is optional with a default, so an agent written against an older version keeps parsing newer observations and the harness keeps parsing older decisions. The optional fields are absent from the schemas' `required` lists by construction. Removing, renaming or retyping a field, making an optional field required, and removing or renaming an action variant are all defined as breaking and forbidden on the v1 surface. A breaking change never mutates the v1 types in place; it ships a parallel namespace with its own schemas and its own major version, and the two run side by side through a published deprecation window. Adding a new action variant is itself classified as breaking, because consumers that exhaustively match would misparse it, so it also goes through the parallel namespace rather than an additive bump.

4.3 Version semantics

`CONTRACT_VERSION` versions the wire shape only and is deliberately decoupled from the crate and package versions, which move on their own release cadence. A major bump is a breaking change shipped as a parallel namespace. A minor bump is reserved for a batch of additive fields significant enough to advertise; a single additive field needs no bump at all, because existing agents are unaffected by definition. A package release never, on its own, bumps the contract version.

4.4 Why the contract can be trusted without trusting the host

The governance promise is social; the properties of Sections 3.1 to 3.3 are technical, and together they remove the host from the trust equation for everything the byte-identity guarantee covers. That coverage has a boundary worth stating where the guarantee is advertised: the Rust core, its WebAssembly and Python bindings, and the engine outputs behind F1 through F3 are pinned to the byte; the Python evidence layer behind the remaining probes is deterministic in its seeds but sits outside the golden-hash path (Section 7). Within the covered surface, a leaderboard entry can be recomputed from the frozen seed and submitted decisions on any tested platform. The scoring rules

are defined independently by the SharpeBench kernel’s deflated Sharpe, reliability and process gates [Toca, 2026, Bailey and de Prado, 2014].

5 Experiments

Ten findings, eight core calibrations plus two reviewer-motivated probes, and three follow-ups address questions the ten left open: an existence witness for rank-eligibility (Section 5.2), an asymmetric positive control for the manipulation probe (Section 5.8) and sealed evaluation seeds (Section 5.13). Each subsection states the entry point, seeds and quantity measured, then reports numbers from JSON and figures committed with its producer. The exact commands are listed in Section A. F1 serializes package version 0.9.0; other legacy artifacts do not all carry a runtime version, so provenance is the containing Git revision rather than a reconstructed tag claim. F4, F5 and the witness were regenerated after the estimator and dimensional corrections in this revision.

Three scoping statements govern the whole section. First, the findings are instrument calibrations on a generator whose canonical tape the realism gate of Section 5.5 declines to certify (2 of 24 runs pass the calibrated three-check conjunction); certification is reported beside the results, not required by them. Second, the byte-identity guarantee covers engine outputs behind F1 through F3 and the canonical linear path; Python evidence reductions are seed-deterministic on one platform but outside the golden-hash path. Third, the core evidence uses the explicitly stated train-band seeds except F3, which exercises the split; the predictability probe uses a separate 50,000-start band and reports why a bounded public band is unsafe. Dispersion is reported throughout: F1 and F3 use 2,000 seed bootstrap resamples, while F2, F5 and F6 use t-based 95% intervals over committed vectors. Selected F5 cells additionally carry a 135-cell Bonferroni familywise interval.

5.1 F1: Pipeline validation under the corrected kernel

The reference-policy field (flat, equal-weight long, momentum, minimum-variance, maximum-Sharpe, fractional-Kelly volatility targeting) is rolled over sixteen scenario seeds in each of the Calm, Hard and Extreme tiers via `run_baselines`. Each policy’s pooled return series is scored by the SharpeBench `score_run` kernel with the declared trial count equal to the field size, and each row carries a seed-paired bootstrap 95% confidence interval on the deflated Sharpe (2,000 resamples).

This experiment validates the producer-scorer pipeline end to end; the scorer-side discovery it reproduces belongs to the companion paper. Under the pre-0.5.0 kernel this board was all zeros: every baseline scored a deflated Sharpe of 0.0000 on every tier. That was not strictness. The kernel applied its annualized 0.5 deflation prior per period without conversion, which set the bar near an annualized Sharpe of 18 on daily bars, a level nothing clears. Finding and fixing that unit bug is the companion SharpeBench paper’s Finding 1 [Toca, 2026] (the derivation: an annualized prior applied per period, corrected by converting once at 252 periods per year); what this paper claims is only that the producer-scorer separation surfaced it, because trajectories produced here could be re-scored under both kernels and the discrepancy attributed to the scorer alone.

Table 1 and Figure 1 show the corrected board. On Calm, drift is free deflated Sharpe: `kelly_vol_target` and `equal_weight_long` saturate the DSR at 1.0000 (bootstrap CIs [0.9798, 1.0000] and [0.8343, 1.0000]) and `min_variance` reaches 0.9849, though with a near-vacuous interval of [0.0264, 1.0000]. Yet nothing on the board is rank-eligible, because no policy passes the per-run gate on every seed; the best pass^k rate is 0.75. On Hard the best DSR collapses to 0.1114 and on Extreme to 0.0243, and the long baseline’s pass^k rate degrades monotonically with stress (0.62, 0.31, 0.06). `momentum` and `max_sharpe` lose money outright on every tier (mean per-bar returns down to -1.6×10^{-3} on Extreme).

The reading: eligibility is a conjunction, and each leg catches what the other misses. Deflation alone would crown drift on Calm; pass^k alone would miss the overfit breadth the deflation discounts. The earlier all-zeros board looked strict but was measuring a unit error. The corrected board is the honest zero: the number a real agent has to beat is a positive, deflated Sharpe that survives every evaluation

Table 1: Baseline leaderboard under the corrected kernel: deflated Sharpe (DSR) and pass^k rate per tier, 16 seeds each. No policy is rank-eligible on any tier: eligibility requires the per-run gate to pass on every seed (a pass^k rate of exactly 1.00), and the best rate is 0.75.

Policy	Calm		Hard		Extreme	
	DSR	pass^k	DSR	pass^k	DSR	pass^k
kelly_vol_target	1.0000	0.75	0.0277	0.31	0.0029	0.12
equal_weight_long	1.0000	0.62	0.1114	0.31	0.0234	0.06
min_variance	0.9849	0.44	0.0024	0.19	0.0243	0.06
flat	0.0007	0.00	0.0007	0.00	0.0007	0.00
momentum	0.0000	0.00	0.0000	0.00	0.0000	0.06
max_sharpe	0.0000	0.06	0.0000	0.00	0.0000	0.00

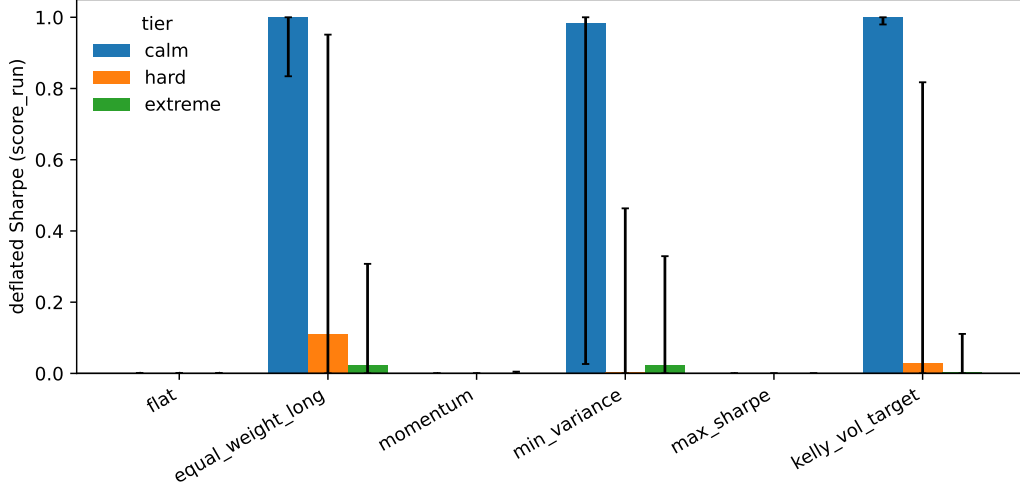


Figure 1: Deflated Sharpe per baseline per tier with seed-paired bootstrap CIs. Calm saturates the long baselines; Hard and Extreme collapse them toward zero.

seed, and no reference policy produces one. One caveat the board cannot answer on its own: no reference policy is rank-eligible, so the board alone does not show that the eligibility set is non-empty for this task family. Section 5.2 answers that by construction, injecting an out-of-band oracle signal of controlled strength and locating the strength at which the gates first open; the region is non-empty on every tier, and the per-run gate that fails every baseline here is the leg that binds there too.

5.2 Rank-eligibility witness: the acceptance set is non-empty

The baseline board of Section 5.1 contains no rank-eligible policy, leaving open whether the conjunction can be satisfied at all. We answer by construction with an out-of-band oracle, not an entrant. For each seed, the script replays the deterministic, policy-invariant tape one bar ahead and mixes standardized next-bar truth z_t with one fixed seeded noise path ε_t :

$$\text{signal}_t(s) = sz_t + \sqrt{1 - s^2} \varepsilon_t, \quad 0 \leq s \leq 1.$$

The same ε_t is reused at every s (common random numbers), so changing signal strength does not silently change the experiment. A sign-following policy trades every bar; a deadband policy trades only when $|\text{signal}| > 1$ and otherwise holds. Both are scored through the unchanged SharpeBench conjunction over 16 seeds. The script first checks the full coarse acceptance set. It refines a crossing by bisection only when coarse eligibility is monotone; otherwise it reports the observed set without calling any crossing a threshold. All ten attained series are monotone after the common-random-number correction.

Table 2: Observed eligibility thresholds in signal-truth correlation s . Intervals are bisection brackets after a monotone coarse-grid check. “None” means that no sampled $s \leq 1$ is eligible. At every attained threshold, the last failing gate is the requirement that every seed’s marginal PSR reach 0.90.

Variant	Band	Calm	Hard	Extreme
sign_follow	held-out	none	[0.7250, 0.7281]	[0.3000, 0.3031]
sign_follow	F1 table	none	[0.7875, 0.7906]	[0.4438, 0.4469]
deadband_hold	held-out	[0.9000, 0.9031]	[0.5031, 0.5063]	[0.3813, 0.3844]
deadband_hold	F1 table	[0.6563, 0.6594]	[0.6188, 0.6219]	[0.3688, 0.3719]

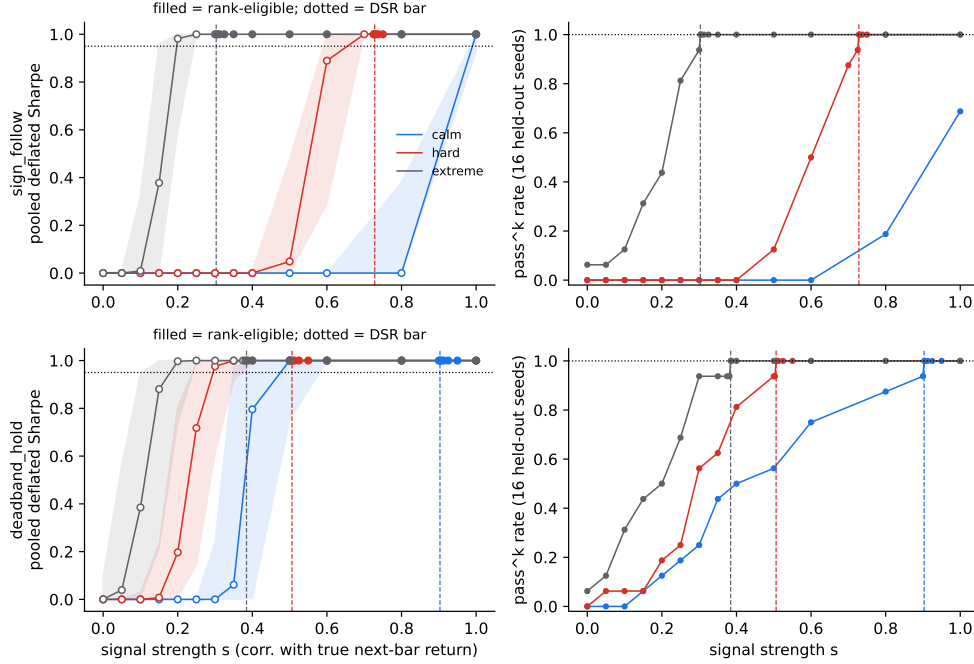


Figure 2: Witness sweep on the held-out band. Filled markers are eligible sampled points; dashed lines mark bisection thresholds only after monotonicity is verified. The right panels show how many of the 16 seed-level PSR tests pass.

The result is an existence proof and a calibration, not evidence about an agent. The deadband construction is eligible on every tier, so the acceptance set is non-empty. The every-seed gate is last to open at all ten observed thresholds; pooled DSR and bootstrap gates open earlier. The sign follower never becomes eligible on Calm, even at $s = 1$, because its every-bar turnover costs more than the low-volatility signal is worth. Thus the environment does not impose an activity requirement, but its cost model does require a Calm policy to ration turnover.

5.3 F2: Regret against the closed-form reference policy

On the Avellaneda-Stoikov market-making task, `mm_regret` runs each candidate quoting policy and the closed-form reference policy (Section 3.5) on the same sixteen seeded episodes ($\sigma = 2.0$, $\gamma = 0.1$, $\kappa = 1.5$, 200 steps) and reports the mean reward gap to the reference. Candidates are the reference itself and fixed-spread quoters over a grid of half-spreads. The evidence commits the per-episode regret vector per grid point; intervals are t-based 95% CIs over the sixteen paired episodes.

The reference’s regret is 0.0 by construction, which confirms the plumbing: the metric’s zero point is the reference, not a claim of unbeatability. The fixed-spread curve is U-shaped (Figure 3): regret is 84.6 (95% CI [68.2, 101.1]) at a half-spread of 0.05 (quoting too tight, filled constantly, run over by inventory), falls to a minimum of 0.037 at 0.5, and rises again to 36.6 ([33.0, 40.2]) at 2.0 and 58.8

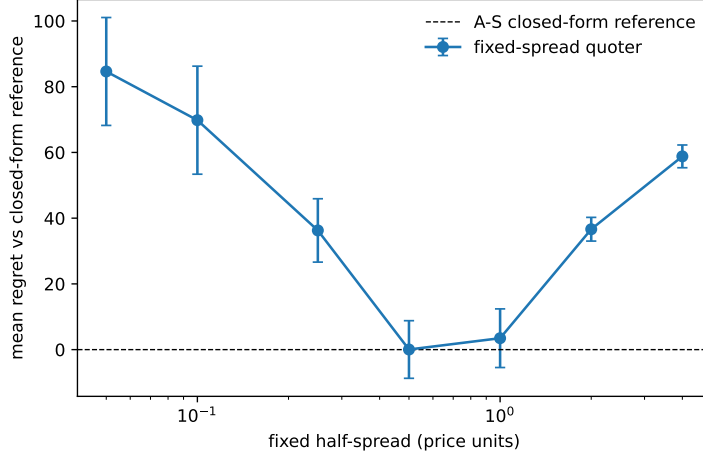


Figure 3: Mean regret versus the analytical Avellaneda–Stoikov reference policy over 16 paired episodes, with t-based 95% CIs. Fixed-spread quoting is U-shaped in the half-spread; the reference defines zero.

Table 3: Left: within-tier generalization gap (train minus test deflated Sharpe) for the fixed equal-weight reference. Right: cross-regime transfer gap (in-distribution minus zero-shot out-of-distribution DSR), rows source regime, columns scored regime; the diagonal is zero by construction, and the Hard↔Extreme cells (± 0.096) sit below the 0.34 seed-noise calibration and are not interpreted.

Tier	Train DSR	Test DSR	Gap		Calm	Hard	Extreme
Calm	1.0000	0.9988	+0.0012	Calm	0.000	+0.877	+0.973
Hard	0.1231	0.4618	−0.3388	Hard	−0.877	0.000	+0.096
Extreme	0.0269	0.1280	−0.1012	Extreme	−0.973	−0.096	0.000

([55.3, 62.3]) at 4.0 (quoting too wide, earning nothing). The minimum deserves its interval: the 0.5 point’s CI is $[-8.7, 8.8]$ and the 1.0 point’s is $[-5.4, 12.4]$. Both intervals include zero, so sixteen episodes do not resolve either point from the analytical reference; this failure to reject zero is not evidence of equivalence. The tight and wide extremes are separated from zero by wide margins. The instrument resolves the shape of the curve, not the sign of its minimum.

The point of shipping the reference is that agents on this task are scored on regret, not raw PnL. A market-making leaderboard ranked on PnL rewards whichever policy drew the friendliest flow; regret against a fixed analytical reference on the same episodes removes that luck axis.

5.4 F3: Generalization gap and cross-regime transfer

`generalization_gap` scores the default reference policy (equal-weight long) on disjoint train and held-out seed bands (16 seeds each, separated by a 10,000-seed gap) within each tier. `cross_regime_transfer` then holds sixteen seeds fixed and varies the regime, producing the full three-by-three transfer matrix whose diagonal is zero by construction, because identical regimes reuse byte-identical environments. One framing note before the numbers: the policy is fixed and parameter-free, so nothing is trained, "source regime" below means the regime the policy is nominally selected on, and for such a policy the matrix reduces to pairwise differences of three per-tier deflated Sharpes. What F3 measures is the instrument, how much score a regime switch moves for a fixed reference, not transfer learning. The evidence commits per-seed return series per cell; CIs are seed-resampled percentile bootstrap ($B = 2000$), seed-paired across regimes for the matrix cells.

Table 3 reports both. The within-tier gaps are the expected control: the reference policy has no fitted parameters, so it cannot overfit its train band, and the instrument correctly reads near zero on

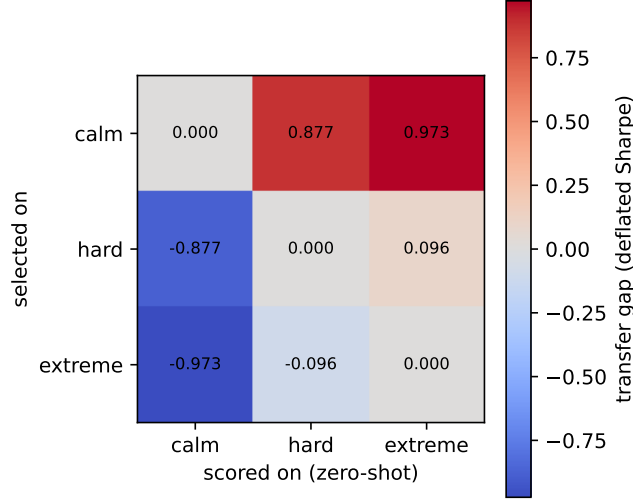


Figure 4: Cross-regime transfer matrix for the reference policy: out-of-distribution deflated Sharpe with the seed band held fixed.

Calm (+0.0012) and noise-signed values on the stressed tiers (the negative Hard and Extreme gaps mean the held-out band happened to be kinder; with no learning, the gap is pure seed noise, and its magnitude, up to 0.34 on Hard, calibrates how much band luck a 16-seed evaluation carries). The per-band bootstrap intervals say the same thing at interval width: Hard’s train-band DSR of 0.1231 carries a CI of $[0.000, 0.944]$ and its test band $[0.037, 0.948]$ (the bands are disjoint seed sets, so these are unpaired), overlapping almost entirely, which is what a pure-noise gap looks like.

The transfer matrix (Figure 4) is where regime structure appears, and its cells split by the paper’s own noise calibration of 0.34. The Calm-involved point estimates are far above it: source regime Calm scored zero-shot on Hard costs 0.877 of deflated Sharpe, and Calm to Extreme costs 0.973, essentially the whole score. The seed-paired bootstrap intervals temper the two claims differently: Calm to Extreme excludes zero decisively ($[0.635, 0.999]$), while Calm to Hard’s interval is wide ($[0.054, 0.999]$), positive throughout but reaching close to zero, so at 16 seeds only the Calm-to-Extreme collapse is resolved beyond doubt. The Hard-to-Extreme cell (0.096, CI $[-0.218, 0.911]$) sits *below* the 0.34 seed-noise calibration and its interval straddles zero, so it is not interpreted: at 16 seeds the instrument cannot distinguish it from band luck. The wide intervals are themselves a calibration result: a bounded, heavily deflated score bootstrapped over 16 seeds is a coarse instrument, and claims narrower than these intervals should not be made from boards this size. The matrix is antisymmetric because the policy is fixed, and its diagonal is zero by construction. Its off-diagonal magnitude is the quantity a within-tier gap is structurally blind to: a policy scored only on calm seeds can carry a saturated DSR into a leaderboard while being worth 0.03 under stress. Both metrics are cheap, and the protocol requires reporting both.

5.5 F4: Stylized-facts realism certification

For each tier, price panels from eight seeded episodes are graded by `certify_realism` against three directional bounds from the classical catalogue: positive excess kurtosis, positive absolute-return autocorrelation and positive aggregational Gaussianity [Cont, 2001]. Gain/loss skew [Cont, 2001], timescale asymmetry [Zumbach, 2009], and a finite-panel Fano exceedance-count ratio are informational. The Fano ratio is not gated: its threshold is estimated from the same 121-bar panel, leaving only 12 count windows, so 1 is not a calibrated finite-sample null even after using an unbiased sample variance.

Table 4: Stylized-facts results over 8 seeds. Fano is an exploratory finite-panel proxy and has no pass column; the final rate is the conjunction of the three calibrated directional checks, including aggregational Gaussianity (not shown).

Tier	Excess kurtosis		$ r $ autocorr		Fano proxy	Gated facts
	mean	pass	mean	pass	mean	pass rate
Calm	-0.97	0/8	+0.0002	5/8	1.30	0.000
Hard	+16.30	8/8	-0.0107	1/8	0.92	0.125
Extreme	+11.38	8/8	-0.0118	1/8	0.86	0.125

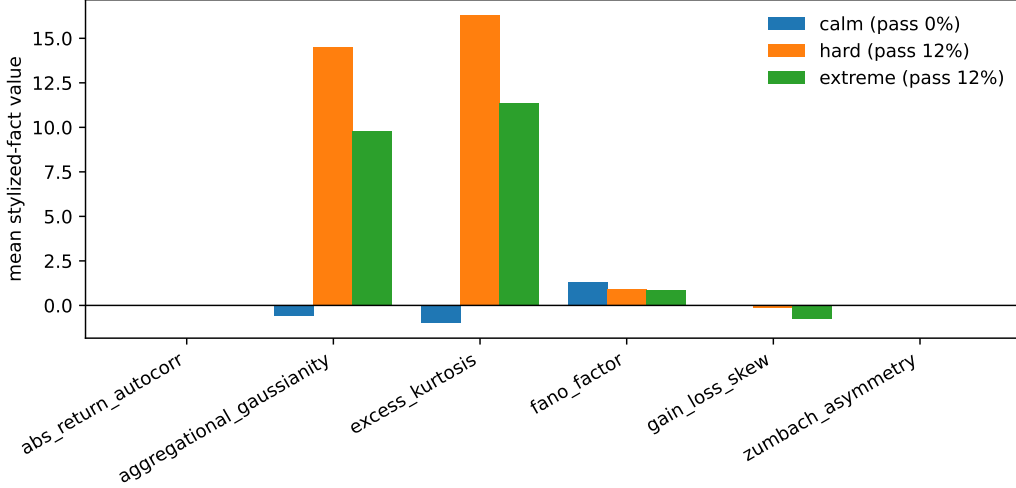


Figure 5: Per-seed stylized-fact values by tier. Only kurtosis, absolute-return autocorrelation and aggregational Gaussianity are gated; Fano is exploratory.

Table 4 and Figure 5 give a mixed verdict. Calm is platykurtic (mean excess kurtosis -0.97 , aggregational Gaussianity -0.58) and fails the conjunction on all eight seeds. Hard and Extreme pass excess kurtosis and aggregational Gaussianity on all eight seeds, but absolute-return autocorrelation is slightly negative at this episode length; each tier passes the three-check conjunction on only 1 of 8 seeds. Thus the canonical generator passes 2 of 24 seeded panels. The informational Fano ratios are reported as observations only. They do not identify why variance concentrates, and they no longer decide certification.

An opt-in `vol_clustering` knob adds deterministic EMA persistence while preserving the zero-knob golden path. On the same diagnostic seeds at strength 0.5, autocorrelation passes rise from 5/8 to 8/8 on Calm, 1/8 to 8/8 on Hard and 1/8 to 5/8 on Extreme. With Fano correctly excluded from the gate, the three-check pass rates become 0/8, 8/8 and 5/8. This is still in-sample calibration: strength 0.5 was tuned and rechecked on the diagnosis panel. The newer opt-in jump-burst knobs are implemented but have no committed calibration experiment here, so neither they nor the exploratory Fano proxy support a remediation claim.

5.6 F5: The manipulation boundary and size response

A push-and-unwind manipulator is scored against its zero-impact paired reference (identical seed and schedule, $\lambda = \eta = 0$) over eight seeds, so `impact_pnl` attributes profit specifically to having moved the price. `impact_boundary_sweep` sweeps permanent impact (Kyle λ), temporary impact (η) and the momentum-follower gain over six grid points per axis; `size_response` sweeps the push weight over five points. The evidence commits per-seed impact-PnL vectors at every grid point; intervals below are t-based 95% CIs over the eight seeds.

Table 5: Manipulation probe, impact-attributable PnL as fraction of capital per episode, in units of 10^{-4} , mean over 8 seeds with t-based 95% CIs. Left: endpoints of each swept axis (6 grid points per axis); every axis is unprofitable everywhere, so no profitability boundary exists to report. Right: the full size-response grid in the push weight.

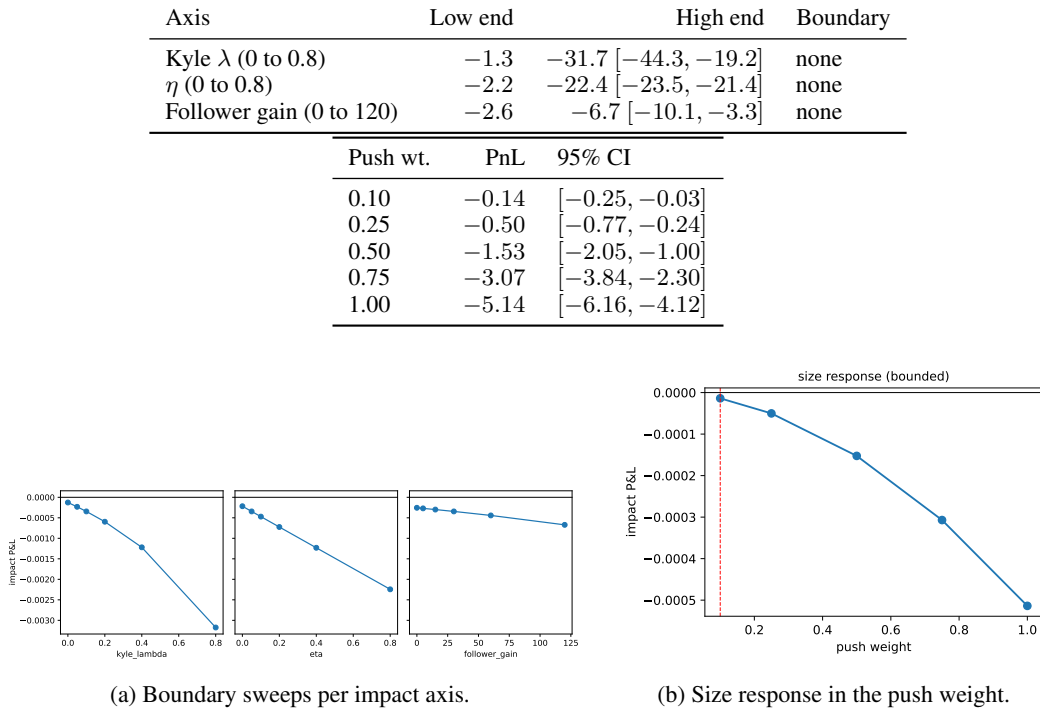


Figure 6: The manipulation probe: impact-attributed PnL is negative everywhere and worsens with impact strength and with size.

Under the precise linear, time-independent permanent-impact assumptions studied by Huberman and Stanzl [2004] and related no-dynamic-arbitrage conditions of Gatheral [2010], round-trip price manipulation is ruled out. The finite linear sweep below is therefore a model-consistency check, not a universal certificate. Section 5.7 introduces dimensionally normalized nonlinear flow, and Section 5.8 reports an exploratory asymmetric search with familywise inference.

The result (Table 5, Figure 6) is consistent with the linear model’s no-manipulation conditions on the sampled grid. On every axis the pump loses money at every sampled coefficient, with every per-point 95% CI excluding zero, including the realistic center of the grid ($\lambda = 0.1$, $\eta = 0.05$: impact PnL -3.4×10^{-4} , CI $[-4.3, -2.6] \times 10^{-4}$). The size response is bounded and decreasing: the least-bad size is the smallest sampled push weight and the loss grows to -5.1×10^{-4} at full size. This finite sweep is not a proof over schedules or parameters outside the grid. The two extensions that follow test what it does not cover. The concave ablation keeps the symmetric schedule and still finds no profit, which is a measurement rather than a corollary; the positive control then finds profitable asymmetric round trips in two selected concave cells after a familywise correction, while all 45 sampled linear cells remain negative. The module’s disclaimer remains explicit: it diagnoses the simulator, not a trading strategy.

5.7 Normalized-flow concavity: a falsifiable symmetric probe

The linear F5 sweep is primarily a consistency check under the model assumptions of Huberman and Stanzl [2004], Gatheral [2010]. To make the probe capable of returning a positive, the engine supplies an opt-in permanent-impact exponent on dimensionless crowd flow $x = Q/V$:

$$I_\kappa(Q; V) = \lambda \operatorname{sign}(x)|x|^\kappa, \quad x = Q/V.$$

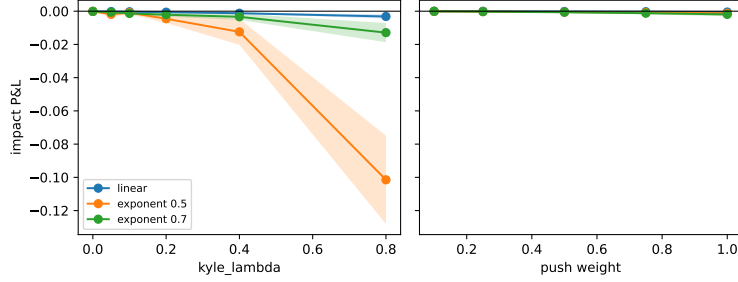


Figure 7: Pointwise impact-attributed P&L intervals for the sampled symmetric schedule under linear and normalized-flow concave impact.

The temporary own-size term remains $\eta q_i/V$. This normalization is load-bearing: applying a power to raw shares and dividing by V afterwards would change units with κ and make results depend on the arbitrary share denomination. A regression test rescales capital, flow and V together and requires the nonlinear impact fraction to remain unchanged. At $\kappa = 1$ the engine uses the original expression $(\lambda Q + \eta q_i)/V$ exactly, preserving every linear golden bit; only $\kappa \neq 1$ enters the powf path outside the cross-runtime golden guarantee.

We rerun the symmetric push-and-unwind sweep at $\kappa \in \{0.5, 0.7\}$, eight seeds per point, with paired zero-impact references and pointwise t intervals. No sampled mean is positive. At the canonical calibration, impact-attributed P&L is -8.9×10^{-4} (95% CI $[-19.6, 1.8] \times 10^{-4}$) for $\kappa = 0.5$, -13.5×10^{-4} (CI $[-21.2, -5.9] \times 10^{-4}$) for $\kappa = 0.7$, and -3.4×10^{-4} (CI $[-4.3, -2.6] \times 10^{-4}$) for the linear arm. For $\kappa = 0.5$, 7 of 23 stored sweep slots have intervals crossing zero; repeated canonical slots reduce these to five distinct configurations. At $\kappa = 0.7$, 0 of 23 cross zero. These are finite-grid, pointwise summaries of this symmetric schedule, not a universal no-manipulation result.

The null identifies the schedule’s limitation rather than closing the economic question: profitable constructions under nonlinear permanent impact can be asymmetric. The exploratory family in Section 5.8 searches such shapes and reports selection-adjusted inference.

5.8 Exploratory asymmetric-schedule positive control

Concavity makes slicing matter. For positive dimensionless flow $x = Q/V$, the sum of the n per-slice impact increments before the engine’s multiplicative price compounding is

$$nI_\kappa(Q/n; V) = n^{1-\kappa} I_\kappa(Q; V),$$

so a slowly accumulated leg and a block liquidation need not cancel when $\kappa < 1$. The identity is about the impact function, not the compounded terminal price. It motivates a positive control; it does not guarantee that one of our finite schedules must profit after compounding, temporary costs and follower flow.

The search crosses five up/down duration ratios, three block fractions, three exponents and three market arms: 135 cells, each on eight seeds. All displayed cell intervals are pointwise. Because the reported winner is selected after seeing the grid, its headline inference uses a two-sided Bonferroni familywise 95% interval over all 135 cells (Student t , 7 degrees of freedom); the evidence stores the family size, critical value and adjusted interval.

The best observed cell is the no-follower, zero-temporary-impact arm at $\kappa = 0.5$: nine bars of uniform accumulation followed by one-bar liquidation yield $+21.8 \times 10^{-4}$. Its pointwise 95% interval is $[17.4, 26.2] \times 10^{-4}$ and its familywise interval remains positive at $[10.0, 33.6] \times 10^{-4}$. The same sampled schedule with temporary impact yields $+18.4 \times 10^{-4}$, with familywise interval $[6.4, 30.4] \times 10^{-4}$. The canonical follower arm has no pointwise-positive cell. At $\kappa = 0.7$ no sampled mean is positive. Among the 45 sampled linear cells, every mean is negative; the 1:9 and 9:1

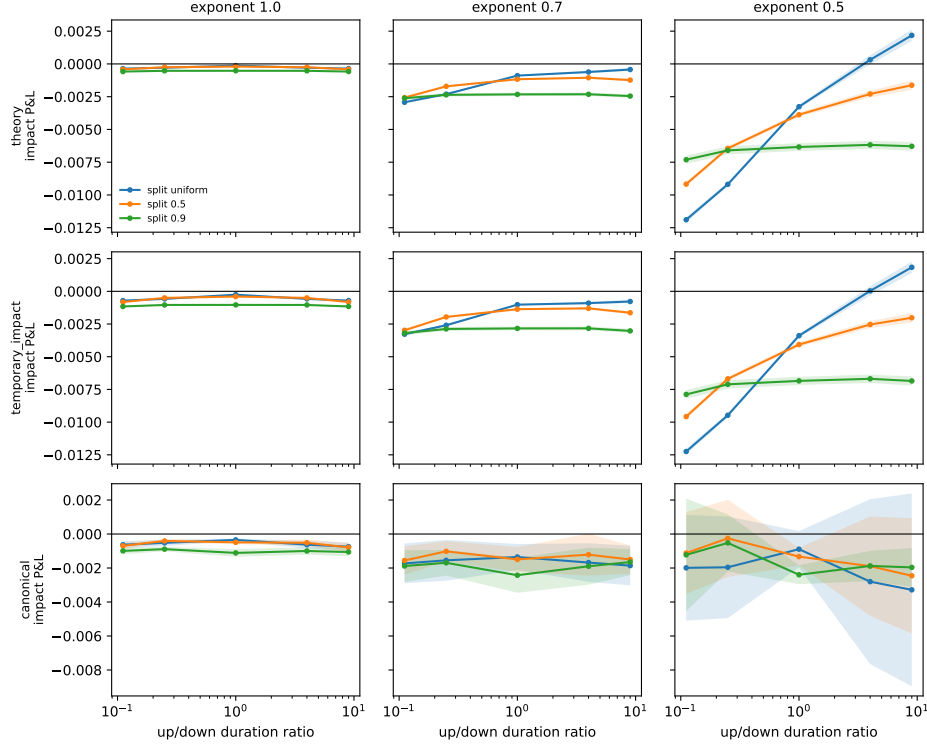


Figure 8: Exploratory asymmetric-schedule search. Shading shows pointwise 95% intervals; selection-adjusted intervals for the two selected $\kappa = 0.5$ winners are reported in the text. Two sampled arms, not one, retain positive familywise intervals.

theory-arm values are numerically near-symmetric, not bit-identical. These statements concern the sampled grid only.

The positive control shows that this instrument can detect a profitable sampled round trip under normalized concave impact. It does not establish a universal region, an optimum, or a theorem-level correspondence: the search fixes push size, λ , V , total leg length and most follower gains, and omits short-side, overshooting and longer-hold trips. Conversely, the absence of a positive among 45 sampled linear cells is consistent with the cited linear-impact model under its assumptions, not empirical proof about every configuration of the product.

5.9 F6: Adverse-selection markout decomposition

`compare_informed_vs_uninformed` runs twenty-four paired episodes in which the identical meta-order flow is sided on the alpha in one leg and on a coin flip in the other, holding schedule, sizes and price path fixed, and reports mean maker markout per filled unit at horizons of 1, 5 and 20 bars. Markout is measured in the tape's absolute price units (the mid starts at 100) per unit of filled quantity; the evidence commits per-episode vectors, and the gap CIs below are t-based 95% over the twenty-four paired episodes.

Two certifications must be separated here: the sign of the *gap* and the sign of the *levels*, and they come out on opposite sides.

The gap certification passes at every horizon (Table 6, Figure 9): makers filled by informed flow mark out worse than the paired-uninformed control at $h = 1, 5$ and 20, the per-unit gap widens from 0.072 to 0.436 as the information realizes, and every horizon's paired CI excludes zero, including the smallest ($h = 1$: [0.052, 0.094] over per-episode paired gaps). That is the signature shape of adverse

Table 6: Maker markout per filled unit (price units per unit quantity) against informed versus paired-uninformed flow, 24 paired episodes; gap CIs are t-based 95% over per-episode paired gaps. The gap is the price of trading against information and grows with horizon; the levels stay positive at every horizon, a separate finding discussed in the text.

Horizon (bars)	Informed	Uninformed	Gap	Gap 95% CI
1	0.689	0.761	0.072	[0.052, 0.094]
5	0.489	0.791	0.302	[0.228, 0.383]
20	0.386	0.823	0.436	[0.303, 0.579]

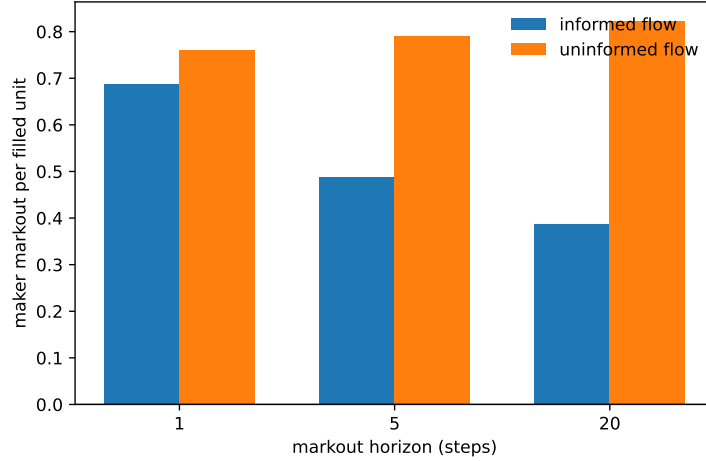


Figure 9: Markout per filled unit by horizon: informed flow versus the paired-uninformed control.

selection: the loss is in the drift after the fill, not in the fill itself. The claim is scoped to the gap, the differential cost of informed over uninformed flow, and not to maker profitability levels.

The level result goes against the environment, and we report it with the same candor as F4. Makers earn a *positive* mean markout against informed flow at every horizon (0.689, 0.489, 0.386 per unit), which is on the wrong side of the Section 2 standard that a realistic market should not let makers profit from informed flow. The cause is calibration and design, not accounting: fills are struck at the pre-move mid, so every fill books the full quoted depth as spread capture, the default quote depths are wide relative to the drift the $\alpha = 2.0$ meta-orders realize over 20 bars, and the price path is exogenous, so quotes never widen in equilibrium response to toxicity as they would in Glosten and Milgrom [1985]. At this calibration the probe therefore measures markout accounting against a fixed drift, in which informed flow costs makers relative to uninformed flow but not in absolute terms. Recalibrating the informed alpha or striking fills at post-impact prices so the level turns negative, and adding a variant where quotes widen with realized toxicity, are the named follow-ups.

The single-episode maker decomposition illustrates the exact identity on one seeded episode (seed 0); it is an illustration, not an aggregate finding. The wider quoter (maker_0, 135 fills) keeps a positive markout at $h = 20$ (0.526 per unit, toxic-fill rate 0.296): the flow is not selecting against it. The tighter quoter (maker_1, 92 fills) shows the subsidized pattern: aggregate markout is still positive at $h = 20$ (0.197 per unit) but its meta-order fills alone lose 0.197 per unit on 62% of its filled quantity, with a toxic-fill rate of 0.533 at $h = 20$, so noise flow is paying for the informed flow. Spread capture and adverse drift decompose exactly (spread_capture + adverse_drift = markout, in price units times quantity): maker_1 captures 82.8 of spread and gives back 64.7 to adverse drift by $h = 20$.

Table 7: Failure-mode counts per tier, 128 episodes each (8 policies $\times$ 16 seeds). No episode reaches bankruptcy or a cascade wipe; failure is dominated by mandate breaches, and drawdown breaches triple from Calm to Extreme.

Tier	Clean	Stopped out	Mandate struct.	Mandate DD	Mandate inv.	Bankrupt	Clean rate
Calm	72	0	36	11	9	0	0.56
Hard	68	0	36	14	10	0	0.53
Extreme	50	5	31	32	10	0	0.39

5.10 F7: Failure-mode distributions

Every rollout of the eight-policy field (the six reference policies plus the two behavioral counterparties), across all three tiers and sixteen seeds (384 episodes), is classified by `classify_episode_failure` into the worst-case-first taxonomy: cascade-wiped, bankrupt, stopped-out, mandate breach (structural, drawdown or inventory), or clean, with a per-scenario sampled mandate and a 50% drawdown stop in force.

One mechanism drives the largest counts and must be stated first: mandates are sampled per scenario and assigned to fixed policies that cannot observe or adapt to them, so a structural breach is guaranteed whenever the draw pairs, say, a shorting policy with a long-only mandate. The flat structural-breach counts across tiers (36, 36, 31) are the signature of that assignment mechanism, not a behavioral finding about agents; a mandate-aware field could invert the distribution entirely, and measuring one is the interesting follow-up.

Table 7 and Figure 10 show the shape of failure as stress rises. The clean rate falls from 0.56 on Calm to 0.39 on Extreme. Structural mandate breaches are roughly flat across tiers, as the mechanism above dictates: a shorting policy under a sampled long-only mandate breaks the rule regardless of the weather. The stress-sensitive modes are the drawdown family: mandate drawdown breaches triple from 11 on Calm to 32 on Extreme, and hard stop-outs at the 50% line appear only on Extreme (5 episodes, four of them `max_sharpe`). No policy in this field ever reaches bankruptcy or a cascade wipe; the taxonomy’s worst states are reachable but not by these baselines at this stop level. Per policy, `flat` is clean 48/48 by construction, while `overconfident` is the worst citizen on Calm (7 of its 16 episodes end in a drawdown breach) and `max_sharpe` is the worst on Extreme (3 clean of 16). The headline, scoped to what was measured: on this mandate-blind reference field, half of everything that goes wrong is process, not PnL, which is exactly the half a return-ranked board never sees. Whether the same holds for agents that can read their mandate is an open measurement, not a claim of this paper.

5.11 F8: Ecology outcomes under shocks, replicated over seeds

`run_ecology` seeds the replicator with the eight baseline species over a shared-book market via `market_payoffs` (12 generations, field size 8, innovation every 4 generations breeding parameter-perturbed variants of the leader) and runs trajectories under a steady all-Calm control schedule and a `regime_shocks` schedule rotating Calm (generations 0 to 3), Hard (4 to 7) and Extreme (8 to 11). The probe is replicated over eight replicator seeds per schedule; Table 8 reports the cross-seed winner distribution, and the seed-0 trajectories (Figure 11) remain the illustrative detail. Note the two schedules chain episode seeds differently (the shock schedule strides seeds across generations), so control and shocked runs at the same replicator seed are matched in configuration, not in price paths.

The replicator flow rule sits in the market-ecology tradition: strategies as species whose profitability decays as their share of the flow grows [Farmer, 2002], and populations whose composition, not any single agent’s optimization, drives aggregate dynamics [Lux and Marchesi, 1999]. The implementation abstracts away much of what those models study: capital flows react instantly (no fund-flow lags), there are no leverage or margin constraints beyond the strategies’ own sizing, and there is no entry beyond the single leader-perturbation innovator. Its outputs are trajectories of this replicator map, not claims about market ecology in general.

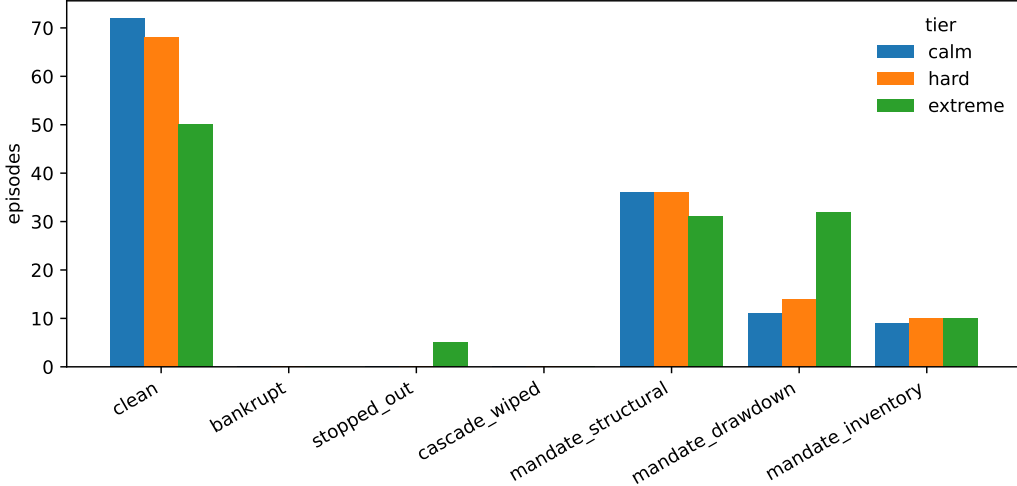


Figure 10: Failure-mode distribution per tier per policy over 384 classified episodes.

Table 8: Replicated ecology outcomes over 8 replicator seeds per schedule: distribution of the final dominant lineage (bred variants credited to their founding species). The root winner differs between the schedules on 5 of 8 seeds; the seed-0 replacement of the volatility targeter by the unlevered long book occurs on 1 of 8 seeds.

Final dominant lineage	Control (all Calm)	Shocked (Calm/Hard/Extreme)
kelly_vol_target	4	7
equal_weight_long	2	1
flat (no resolved dominant)	2	0

The replication overturns the single-seed narrative, and we report that plainly. At seed 0 the story was clean: `kelly_vol_target` sweeps the control board (peak share 1.00, final 0.81), collapses under the shock schedule once the regime turns Hard, and `equal_weight_long` inherits dominance (final 0.67). Across eight seeds that story is the exception. Under the shock schedule a `kelly_vol_target` lineage ends dominant on 7 of 8 seeds (final shares near 0.90); the seed-0 handoff to the long book happens once. The control side is itself seed-dependent: `kelly_vol_target` wins 4 seeds, `equal_weight_long` wins 2, and on 2 seeds no species resolves dominance at all (all eleven end persistent at small shares, with a `flat` variant holding the largest share at 0.10). The root-level winner differs between the two schedules on 5 of 8 seeds, so shocks do reorder outcomes more often than not, but with no consistent direction. Extinction pressure is more stable than identity: shocked runs produce a resolved dominant on 8/8 seeds with 4 to 7 extinctions per run (at seed 0, both behavioral counterparties are gone or collapsed on both schedules, `overconfident` by generation 1).

The honest conclusion is methodological. One replicator trajectory is one draw from a heavy-tailed outcome distribution, and single-run ecology narratives, including the one this paper’s earlier draft promoted to the abstract, do not survive replication. What the probe reliably produces at this scale is the outcome distribution itself: which lineages can win, how often shocks reorder the board, and how much of the population goes extinct, per committed seed set. Sharper claims need more seeds, varied shock orderings and path-matched schedule pairs, all of which are mechanical extensions of the committed script.

5.12 Predictability probe: measuring the in-band inversion attack

Section 7 names a threat the leak-free interface property does not cover: a scenario is a pure function of a public 64-bit seed under a public deterministic transform, so future bars are in principle computable from past bars, and no forbidden operation is needed. This probe converts that named threat into

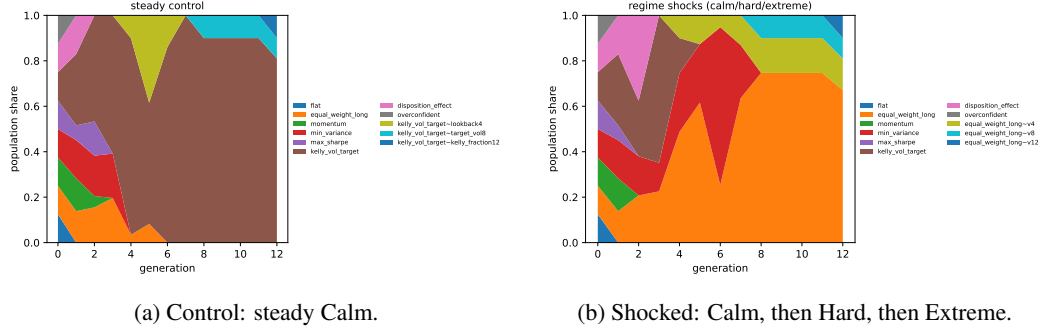


Figure 11: Seed-0 population-share trajectories over 12 replicator generations, the illustrative single run behind Table 8’s distribution. At this seed the regime rotation replaces the calm-regime winner with the unlevered long book; across the eight-seed replication that replacement occurs once.

a measurement. Three adversaries predict the next bar’s return from a causal prefix only, on the three volatility tiers, 16 seeds per tier (4 symbols, 120 days, 30 warmup bars): an *unconditional baseline* that predicts the expanding prefix-mean return per symbol; an *honest statistical adversary*, a ridge AR(5) refit on the prefix at every bar, with no knowledge of the generator; and a *known-seed oracle*, which replays the generator from the true seed and therefore predicts every future bar exactly, the upper bound on any inversion attack. Each predictor is also run as a frictionless unit-gross long/short sign-following policy and scored through the public `score_run` kernel; we report the kernel’s deflated-Sharpe probability.

The honest adversary extracts almost nothing. All “ $\pm$ ” quantities here are across-seed standard deviations, not confidence intervals. On Calm the ridge AR reaches $55.4\% \pm 2.8\%$ directional accuracy against the baseline’s $54.1\% \pm 2.4\%$: it detects the tier’s designed momentum autocorrelation, worth about one descriptive accuracy point over drift-following. On Hard and Extreme the AR accuracies are $51.2\% \pm 3.3\%$ and $50.6\% \pm 2.1\%$, with MSE no better than the unconditional baseline (Hard: 1.24×10^{-4} vs. 1.22×10^{-4} ; Extreme: 1.27×10^{-3} vs. 1.17×10^{-3}). No hypothesis test or equivalence margin was preregistered, so these summaries support “near the unconditional baseline,” not statistical indistinguishability from 50 percent and not proof that no edge exists.

The oracle bound is total, and a bounded seed search reaches it. The known-seed oracle scores 100% accuracy, zero MSE, and DSR 1.00 on every tier: determinism means the gap between an honest adversary and a successful inverter is the entire distance from coin flipping to omniscience, so the question is only the cost of inversion. We implemented the strongest cheap version: a brute scan over a bounded seed band of width 2^{16} centered on the published `start_level`, matching each candidate’s first-bar closes against the single observed opening bar. The scan recovered the true seed in 16 of 16 trials with zero collisions, from a prefix of one bar, at $14.7 \mu\text{s}$ per candidate (≈ 0.96 s per scan); every recovered seed was then verified to reproduce the full 30-bar observed prefix on its tier. The lesson is sharp: when the deployed seed band is bounded and public, as a `(start_level, num_levels)` spec makes it, in-band inversion is not cryptanalysis but a table scan, and any such deployment should treat its scenarios as fully known to a motivated adversary. Extrapolating the measured per-candidate cost to the unbounded 2^{64} space gives ~ 8.6 million CPU-years, so brute-force inversion of an unpublished seed drawn from the full space is infeasible by enumeration.

What remains open. The probe brackets the attack but does not close it. Between the honest regression (no edge beyond disclosed structure) and the band scan (total recovery given a bounded public band) lies the case that matters for unbounded deployments: analytic inversion of SplitMix64 through the `mul/add/div/max` price transform from observed reals. We implemented and tested the narrow algebraic step: SplitMix64’s 64-bit finalizer is exactly invertible when its complete

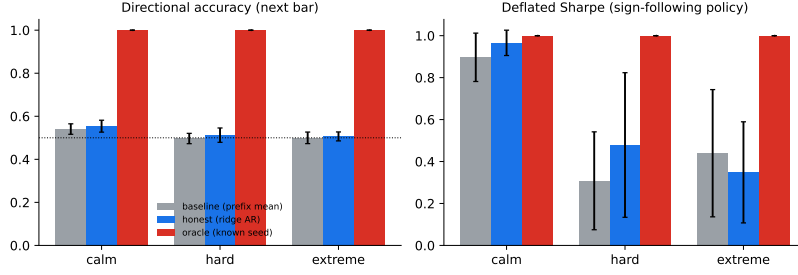


Figure 12: Next-bar directional accuracy by tier. The causal AR adversary remains near the unconditional baseline; knowing or recovering the seed gives perfect prediction.

output word is known. The generator’s `next_unit`, however, exposes only the upper 53 bits as a float, leaving 2^{11} candidate post-increment states from one exact unit observation before any price arithmetic is considered. The public interface reveals neither that unit nor an unrounded finalizer output: it reveals prices after data-dependent arithmetic, floors and serialization. Deciding among those candidates from observed prices remains the untested cryptanalytic step; the new primitive reduces the question, but does not claim a tape-level break. The honest reading of the probe therefore remains that unpredictability rests on seed secrecy and entropy, not transform hardness, and that published bounded seed bands forfeit both.

5.13 Sealed evaluation seeds: defeating the bounded-band scanner

The public held-out set is disjoint from training but enumerable. The band scanner of Section 5.12 therefore recovers its scenario identities without violating the observation interface. The opt-in mitigation derives each named slot from a secret salt k :

$$\text{seed}_i = \text{EVAL_SEED_BASE} + (F(k, i) \bmod (2^{64} - \text{EVAL_SEED_BASE})).$$

The shipped F combines FNV-1a/64 with public SplitMix64 finalizers. It is deterministic and pinned across Rust and Python, and the tested salts and slots produce held-out-band seeds distinct from the public set. No universal injectivity claim is possible for a finite 64-bit mapping, and this construction is PRF-style rather than a certified MAC.

The intended workflow is commit–reveal. Before evaluation, the host publishes $\text{SHA-256}(k)$ and withholds k ; after scoring, it reveals k so anyone can recompute the named seeds and replay the run. SharpeBench’s forward-window format can now bind that hash before entries arrive. This paper’s evidence is a simulated workflow containing the revealed salt, not proof that a live evaluation used secret custody, a non-reuse policy or a reveal deadline; the already-open companion window does not claim sealed evaluation.

On 16 tested slots, the scanner over the old public 2^{16} band recovers 16/16 public seeds and 0/16 sealed seeds while the salt is withheld; revealing the salt replays 16/16. Under a uniform-seed approximation, that scanner covers about 3.6×10^{-15} of the derived band. The result is therefore narrow: high-entropy salt secrecy defeats this particular bounded-band enumeration budget. It is not a cryptographic security proof, and it does not answer tape-level inversion after observations begin.

6 Related work

Environment interfaces. Gym [Brockman et al., 2016] demonstrated that a small, stable environment interface is worth more than any single environment, and Gymnasium [Towers et al., 2024] carried the interface forward with versioned environment IDs and a conformance checker; SharpeArena’s Python surface is a first-class Gymnasium citizen with versioned, namespaced IDs and adopts its `check_env`. PettingZoo [Terry et al., 2021] standardized the multi-agent case, which SharpeArena’s shared-book and batched-competition environments implement. Minari [Farama

Foundation, 2024] standardizes offline-RL datasets; SharpeArena exports leak-safe rollouts to it. The wire contract of Section 4 differs from all of these in scope: it is a language-agnostic JSON boundary for external agent processes, governed like a protocol rather than a library API.

Evaluation rigor in RL. The environment’s evaluation stance imports three lessons from the RL evaluation literature. From the ALE revisitation [Machado et al., 2018]: determinism in the environment invites trajectory memorization, so evaluation must be designed against it; SharpeArena keeps determinism for verifiability and defeats memorization with procedural scenario distributions and held-out seed bands instead of injected stochasticity. From Procgen [Cobbe et al., 2020]: procedural generation over integer seed intervals makes generalization measurable, which Section 3.4 ports to markets. Prioritized Level Replay [Jiang et al., 2021] shows seed-level curricula shape what agents learn, which motivates keeping the train band operator-owned and the held-out band frozen. τ -bench [Yao et al., 2024] argues that agent reliability must be measured across repeated runs rather than on average; that requirement lives in the SharpeBench kernel this environment feeds [Toca, 2026].

Financial RL environments. A Gym-for-finance lineage predates this work, and SharpeArena should be read against it on the four property axes it competes on: leak-freedom mechanism, determinism guarantee, trajectory verifiability, and the external-agent contract, with the generalization protocol as a fifth. FinRL [Liu et al., 2020] and FinRL-Meta [Liu et al., 2022] wrap historical market data into gym-style training environments at scale; FinRL-Meta’s DataOps pipeline explicitly targets the data-quality half of the problem (survivorship bias, backtest overfitting). Because their environments replay recorded data inside a general-purpose Python process, leak-freedom is a matter of pipeline discipline rather than interface shape, and neither system claims byte-determinism across platforms or third-party replay verification of submitted trajectories; their agent surface is the in-process Gym API rather than a language-agnostic wire contract. ABIDES-Gym [Amrouni et al., 2021] wraps the ABIDES discrete-event market simulator [Byrd et al., 2020] in Gym environments, bringing multi-agent market microstructure to RL training; its focus is simulation fidelity, and to our knowledge it does not target cross-runtime byte-determinism or tamper-evident trajectories. TradeMaster [Sun et al., 2023] is a holistic platform spanning data, environments, algorithms and evaluation for RL trading research; it standardizes the workflow rather than hardening the producer, and its evaluation toolkit runs inside the same trust domain as the agent. `mbt_gym` [Jerome et al., 2023] provides model-based Gym environments for limit-order-book problems, including the same Avellaneda-Stoikov family as Section 3.5, with closed-form solutions available as baselines; it is the closest relative of the market-making task specifically, and Section 3.5 notes the relation. Where these descriptions rest on our reading of the systems’ papers and repositories rather than on properties their authors claim, we have kept them descriptive; none of these systems, to our knowledge, sets out to make look-ahead structurally unavailable at the interface, pin cross-runtime byte-identity with committed hashes, or make submitted trajectories third-party recomputable, which is the specific combination this paper contributes.

Market simulation. ABIDES [Byrd et al., 2020] is the closest simulator in spirit: a high-fidelity multi-agent market with a matching engine and background agent populations. SharpeArena trades some of that fidelity for properties ABIDES does not target: cross-runtime byte-determinism with committed golden hashes, a structurally leak-free observation boundary, replay-based tamper evidence, and a governed external-agent wire contract. The microstructure models are the classical ones: Kyle’s permanent impact [Kyle, 1985], Almgren–Chriss temporary impact and execution cost [Almgren and Chriss, 2001], and the Avellaneda–Stoikov analytical quoting reference [Avellaneda and Stoikov, 2008]. The realism diagnostic compares the generator with selected stylized facts from Cont [2001].

Scoring. The environment deliberately contains no scorer. The deflated Sharpe [Bailey and de Prado, 2014] and the reliability and process gates that consume this environment’s trajectories are specified in the SharpeBench paper [Toca, 2026]; this paper is the producer half of that funnel.

7 Limitations

One synthetic generator family. All three tiers come from a single procedural vol-and-jump generator; no real price series enters the environment, and the tiers differ in that generator’s parameters, not in kind. F4 quantifies the cost: the tape has fat tails on the stressed tiers but weak volatility clustering at episode length, and the calibrated three-check conjunction passes on only 2 of 24 canonical runs. The Fano ratio is exploratory and no longer gates. An agent that excels here has demonstrated skill against this generator’s structure, not against a market, and the realism gate says so rather than hiding it. This is why the abstract and Section 1 frame every finding as instrument calibration on uncertified tape; the framing is a constraint on the claims, not a footnote to them.

The interface guarantee is not an information guarantee. The leak-freedom of Section 3.1 is a statement about the interface’s shape: no operation returns a post-cursor bar. It is not a claim that public deterministic scenarios are unpredictable. Section 5.12 makes the distinction empirical. A causal AR(5) adversary stays near its unconditional baseline, but an exhaustive scan of a public 2^{16} seed band recovers all 16 tested seeds from one observed bar in about one second. Full-space brute force remains infeasible. A complete 64-bit SplitMix64 output inverts to one state, whereas its published 53-bit unit representation leaves 2^{11} candidates; tape-level inversion through the later price arithmetic remains open. The opt-in sealed derivation of Section 5.13 defeats this specific bounded-band scanner 0/16 when its salt is withheld, and the companion SharpeBench arena can commit the salt hash before entry. The evidence here is a simulated workflow, however, and the currently open companion window does not claim to use it. The derivation is PRF-style, not a certified MAC, so its security rests on salt entropy and secrecy rather than a cryptographic proof. "Leak-free" therefore means exactly the interface property and no more.

The baselines are trivial policies, not trained agents. Every number in Section 5 is produced by fixed reference policies (buy-and-hold analogs, one-step tilts, closed-form allocators) and two deliberately biased behavioral counterparties. They establish the honest zero and calibrate the instruments; they do not establish what a trained agent scores, and this paper’s claims are scoped to environment and protocol for that reason, with compute as the stated constraint. In particular, the near-zero within-tier generalization gaps in F3 are the expected control for policies with nothing to overfit, not evidence that the environment resists overfitting by learners. Relatedly, no reference policy is rank-eligible under the kernel’s full conjunction (Section 5.1). Section 5.2 establishes that the eligibility set is non-empty on every tier, but it does so with an out-of-band oracle that replays the seed one bar ahead, which is not an entrant and is not scored on any board; no policy that reads only the observation interface has yet been shown eligible.

Named future experiments. Two follow-ups remain specific. (1) A learning baseline: PPO trained through the shipped Gymnasium adapter on the train band, evaluated held-out and cross-regime, so F3 contains a policy that can actually overfit. (2) Tape-level analytic state recovery from quantized SplitMix64-derived prices. The exact-finalizer inverse and the resulting 2^{11} candidates per 53-bit unit are now tested, but no observed price has been inverted; sealed seeds (Section 5.13) remove the bounded-band attack but do not answer this one. Two named earlier are done. The manipulation positive control left by Section 5.7 is run in Section 5.8: an asymmetric schedule search finds a profitable round trip at $\kappa = 0.5$ and none under linear impact, though the search did not cover push size, λ , legs longer than ten bars, holds longer than one bar or short-side trips. The rank-eligibility existence question of Section 5.1 is settled by the witness of Section 5.2, by an oracle rather than by an entrant.

The Python probe layer is outside the golden-hash guarantee. The byte-identical claim covers the Rust core and its WebAssembly and Python bindings, where golden hashes and parity tests enforce it. The Python reductions that produce the realism, manipulation, adverse-selection, failure, ecology and predictability evidence are deterministic in their seeds, but are off the golden-hash path; their floating-point reproducibility is the platform’s, not the paper’s. The concave-impact engine itself

is also excluded from the canonical byte-identity claim because a general exponent uses `powf`; the linear default remains on the untouched golden path.

The impact models are stylized. The endogenous market composes Kyle and Almgren-Chriss terms; the adverse-selection module does not separate a meta-order’s information from its impact; the follower population is a one-parameter caricature. The concave ablation is not a universal test: its dimensionless flow $|Q/V|$ sits two to three orders of magnitude below the unit crossover, so exponents below one amplify impact relative to the linear arm at the tested calibration, and the symmetric schedule does not search the asymmetric round trips nonlinear theory permits. Section 5.8 does search them and finds one that pays at $\kappa = 0.5$ with no followers, so the symmetric null says this pump does not pay here, and the positive control says the instrument can see one that does; what neither covers is push size, λ , longer legs, longer holds or short-side trips.

The guard is a deny-list. The structural guarantee covers the environment’s own surface. LookaheadGuard extends it to harness tools by pattern, and a deny-list is exactly as good as its patterns; a novel leak path through a side channel the environment does not own, wall-clock time, filesystem state, a network call, is out of scope for both layers. The replay check verifies decisions against frozen data and therefore catches falsified results, not information an agent obtained elsewhere.

No live external entrants yet. The contract, the conformance kit and the replay verifier exist, and the evidence run exercises them, but no third party has yet submitted an agent. A hosted intake, a live leaderboard and multi-tenant isolation of untrusted agent processes do not exist, and the governance document is a written promise backed by tests that has not been stressed by a hostile or careless external implementer. The significance claim of this paper is therefore the verifiability infrastructure that is checkable today; adoption is future work, not an assumption.

Broader impact. Two dual-use surfaces deserve an explicit norm. First, leaderboard scores: a byte-verifiable score is more persuasive, not more externally valid, and a verified SharpeArena number is a claim about a synthetic environment only. SharpeArena scores are not performance marketing, and presenting them to retail audiences as evidence of real-market skill would be a misuse the project’s documentation states as out of bounds. Second, the manipulation probe: it is a payoff-search harness over manipulation schedules, which is dual-use by construction. We judge publishing it net-positive because the linear-impact arm is a diagnostic against the precise no-manipulation assumptions stated in Section 5.6, not a search for deployable trades; all 45 sampled linear configurations lose money, but the paper makes no guarantee outside that finite grid. The module carries its simulator-diagnostic disclaimer in code, and the crude, legible schedules it implements are the ones surveillance already catches in real markets, so the probe teaches defenders more than attackers. The profitable concave cells use sliced accumulation and block liquidation, a schedule family motivated by the published manipulation literature rather than a claim about an executable market strategy.

8 Reproducibility statement

The environment is a Rust workspace of three crates under the MIT and Apache-2.0 licenses, published to crates.io, npm and PyPI, with a pinned toolchain and a lockfile, depending on the published SharpeBench engine crates rather than a vendored copy. The core forbids unsafe code. The determinism claims are enforced rather than asserted: committed FNV-1a golden hashes pin the generated scenarios and the matching engine’s fill tape, the WebAssembly crate’s tests assert byte-identity with the native engine for both stepping and scenario generation, and the npm package’s smoke test performs the replay-and-tamper check of Section 3.3 on every run. The contract artifacts, schemas, conformance fixtures and the governance document, are committed beside the code they govern, and a conformance test exercises the fixtures. Continuous integration covers the Rust surface with formatting, lints as errors, tests and a WASM target build, plus dependency auditing, the npm package and the Python wheel.

Every number in Section 5 is produced by a committed script under `paper/src/` from fixed seeds, writing JSON evidence to `paper/evidence/` and figures to `paper/figures/`; the commands are listed in Section A. The scripts call the package’s public Python API. Reproduction therefore requires the source revision recorded by the evidence manifest, its locked dependencies and the listed commands; a package version alone is not a complete environment specification.

No large language model is a component of the environment, the contract or any reported result. The environment has no judge. Language models were used to draft and edit prose; every bibliography entry was checked against its source.

References

- Franklin Allen and Douglas Gale. Stock-price manipulation. *The Review of Financial Studies*, 5(3): 503–529, 1992.
- Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2): 5–39, 2001.
- Selim Amrouni, Aymeric Moulin, Jared Vann, Svitlana Vyetenko, Tucker Balch, and Manuela Veloso. ABIDES-Gym: Gym environments for multi-agent discrete event simulation and application to financial markets. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF)*, 2021.
- Marco Avellaneda and Sasha Stoikov. High-frequency trading in a limit order book. *Quantitative Finance*, 8(3):217–224, 2008.
- David H. Bailey and Marcos López de Prado. The deflated sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym, 2016.
- Eric Budish, Peter Cramton, and John Shim. The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015.
- David Byrd, Maria Hybinette, and Tucker Hybinette Balch. ABIDES: Towards high-fidelity multi-agent market simulation. In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS)*, 2020.
- Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- Rama Cont. Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.
- Farama Foundation. Minari: A dataset API for offline reinforcement learning. <https://minari.farama.org>, 2024.
- J. Doyne Farmer. Market force, ecology and evolution. *Industrial and Corporate Change*, 11(5): 895–953, 2002.
- Jim Gatheral. No-dynamic-arbitrage and market impact. *Quantitative Finance*, 10(7):749–759, 2010.
- Lawrence R. Glosten and Paul R. Milgrom. Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1):71–100, 1985.

- Olivier Guéant, Charles-Albert Lehalle, and Joaquin Fernandez-Tapia. Dealing with the inventory risk: A solution to the market making problem. *Mathematics and Financial Economics*, 7(4): 477–507, 2013.
- Gur Huberman and Werner Stanzl. Price manipulation and quasi-arbitrage. *Econometrica*, 72(4): 1247–1275, 2004.
- Joseph Jerome, Leandro Sánchez-Betancourt, Rahul Savani, and Martin Herdegen. Mbt-gym: Reinforcement learning for model-based limit order book trading. In *Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF)*, 2023.
- Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.
- Albert S. Kyle. Continuous auctions and insider trading. *Econometrica*, 53(6):1315–1335, 1985.
- Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance, 2020. Deep RL Workshop, NeurIPS 2020.
- Xiao-Yang Liu, Ziyi Xia, Jingyang Rui, Jiechao Gao, Hongyang Yang, Ming Zhu, Christina Dan Wang, Zhaoran Wang, and Jian Guo. FinRL-Meta: Market environments and benchmarks for data-driven financial reinforcement learning. In *Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track*, 2022.
- Thomas Lux and Michele Marchesi. Scaling and criticality in a stochastic multi-agent model of a financial market. *Nature*, 397:498–500, 1999.
- Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.
- Shuo Sun, Molei Qin, Wentao Zhang, Haochong Xia, Chuqiao Zong, Jie Ying, Yonggang Xie, Lingxuan Zhao, Xinrun Wang, and Bo An. TradeMaster: A holistic quantitative trading platform empowered by reinforcement learning. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.
- Jordan Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall Williams, Yashas Lokesh, and Praveen Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- Tiberiu Toca. SharpeBench: A Luck-Robust Benchmark for Trading Agents. Preprint, <https://github.com/general-liquidity/sharpebench>, 2026. Version 0.8.0.
- Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2024.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.
- Gilles Zumbach. Time reversal invariance in finance. *Quantitative Finance*, 9(5):505–515, 2009.

A Commands that produce every number

All commands run from the repository root with the `sharpearena` Python package installed. F1 serializes `sharpearena` 0.9.0. Several legacy evidence files do not serialize a runtime version, so this paper does not reconstruct one from their creation dates; where present, they record `SharpeBench` 0.5.0. The current F4, F5 and witness files were regenerated from the source tree containing the estimator and dimensional corrections reported here. Each script fixes and serializes its seeds, and the evidence manifest records file hashes and producer commands.

Content provenance.

```
python paper/src/make-provenance.py
-> paper/evidence/provenance.json
```

The manifest records the Git parent, a content hash over the Rust, Python, manifest and producer-source snapshot, and the SHA-256 digest of every JSON and figure artifact. The content snapshot, rather than a guessed package label, identifies the exact candidate used for the regenerated evidence.

The design-property checks (Sections 3.1 to 3.3 and 4).

```
cargo test -p sharpearena          # golden hashes, leak-free
                                   # property tests, conformance kit
cargo test -p sharpearena-wasm     # native == wasm parity + golden
npm test --prefix npm/sharpearena  # replay + tamper detection
python -m pytest crates/sharpearena-py # Python surface, check_env,
                                   # seed-band disjointness
```

F1: baseline leaderboard (Section 5.1).

```
python paper/src/make-f1-baselines.py
-> paper/evidence/f1-baselines.json, paper/figures/f1-baselines.pdf
```

The script passes the bootstrap parameters explicitly ($B = 2000$ percentile resamples, $\alpha = 0.05$, resample seed `0x5BA7_2026`) and serializes them in the evidence config; they equal the library defaults defined in `crates/sharpearena-py/python/sharpearena/confidence.py` (`DEFAULT_N_BOOT`, `DEFAULT_RESAMPLE_SEED`), which reuse the scoring kernel’s canonical bootstrap seed.

F2: regret vs the analytical reference (Section 5.3).

```
python paper/src/make-f2-regret.py
-> paper/evidence/f2-regret.json, paper/figures/f2-regret.pdf
```

F3: generalization gap and cross-regime transfer (Section 5.4).

```
python paper/src/make-f3-generalization.py
-> paper/evidence/f3-generalization.json,
   paper/figures/f3-transfer-matrix.pdf
```

F4: stylized-facts realism certification (Section 5.5).

```
python paper/src/make-f4-realism.py
-> paper/evidence/f4-realism.json, paper/figures/f4-realism.pdf
```

F5: manipulation boundary and size response (Section 5.6).

```
python paper/src/make-f5-manipulation.py
-> paper/evidence/f5-manipulation.json,
    paper/figures/f5-boundaries.pdf, paper/figures/f5-size-response.pdf
```

The same command also runs the concave-impact extension at $\kappa = 0.5$ and 0.7 , serializes it under `concave`, and writes `paper/figures/f5-concave.pdf` (Section 5.7). It then runs the asymmetric positive control (Section 5.8): 135 grid points over five duration ratios, three block fractions and $\kappa \in \{1.0, 0.7, 0.5\}$ on the theory, temporary-impact and canonical arms, eight seeds each, serialized under `positive_control` with its config (including the `not_searched` list) and written to `paper/figures/f5-positive-control.pdf`. The linear and concave keys of the evidence file are byte-identical to their previous values.

Rank-eligibility existence witness (Section 5.2).

```
python paper/src/make-witness.py
-> paper/evidence/witness.json, paper/figures/witness.pdf
```

The command replays each seed one bar ahead to recover the true next-bar return (asserting that the canonical tape is policy-invariant), mixes it with one seeded standardized noise path reused at every strength, scores `sign_follow` and `deadband_hold` through `score_run` on every seed and on the pooled series, and first checks eligibility is monotone over the 13-point coarse grid. It bisects only a monotone first crossing, to a resolution of 0.005 in s , on the held-out band and the F1 table band. The JSON records every strength's per-seed PSR, pooled DSR, bootstrap p -value, eligibility flag, monotonicity check and any bracketed threshold behind Table 2.

Sealed evaluation seeds (Section 5.13).

```
python paper/src/make-sealed-seeds.py
-> paper/evidence/sealed-seeds.json
```

The command derives 16 named slots publicly (the eight committed offsets plus eight drawn from the same 2^{16} band) and sealed (the same slots under a 32-byte salt from the operating system, committed by SHA-256 before the run), re-runs the predictability probe's band-scan adversary unchanged against both, then reveals the salt and replays the sealed scenarios against the recomputed seeds. The JSON records the commitment, the revealed salt, the table-build time and every per-trial recovery and replay record.

F6: adverse-selection markouts (Section 5.9).

```
python paper/src/make-f6-adverse-selection.py
-> paper/evidence/f6-adverse-selection.json,
    paper/figures/f6-markouts.pdf
```

F7: failure-mode distributions (Section 5.10).

```
python paper/src/make-f7-failures.py
-> paper/evidence/f7-failures.json, paper/figures/f7-failures.pdf
```

F8: ecology under shocks, 8 seeds per schedule (Section 5.11).

```
python paper/src/make-f8-ecology.py
-> paper/evidence/f8-ecology.json,
    paper/figures/f8-ecology-control.pdf,
    paper/figures/f8-ecology-shocked.pdf
```

The evidence carries the full seed-0 reports (the figures' source) plus, under `multi_seed`, per-seed winners, outcome counts and the cross-seed winner distributions behind Table 8.

Generator predictability (Section 5.12).

```
python paper/src/make-predictability.py  
-> paper/evidence/predictability.json,  
   paper/figures/predictability.pdf
```

The command evaluates baseline, causal AR(5) and true-seed oracle predictors on 16 seeds per tier, then times exhaustive recovery in a 2^{16} candidate band. The JSON contains every per-seed score and the measured search timing.

Dispersion layers. The F2, F3, F5 and F6 scripts serialize the per-episode or per-seed vectors behind every reported interval into their evidence JSONs (`per_episode_regret`, `per_seed_returns` with `band_dsr_ci` and `transfer_gap_ci`, `dispersion`, and `per_episode_markout_per_unit` with `gap_stats` respectively), each with its CI convention recorded in the file, so every interval in Section 5 is recomputable from the committed evidence.

Regenerating every figure from the committed evidence.

```
python paper/src/make-figures.py
```

Each `make-f*` script produces its own figure at run time; `make-figures.py` re-renders all figures from the JSON already in `paper/evidence/` without re-running any experiment, so a reader can check that no figure contains a number the evidence does not.